WolfCafe - Use Case 12

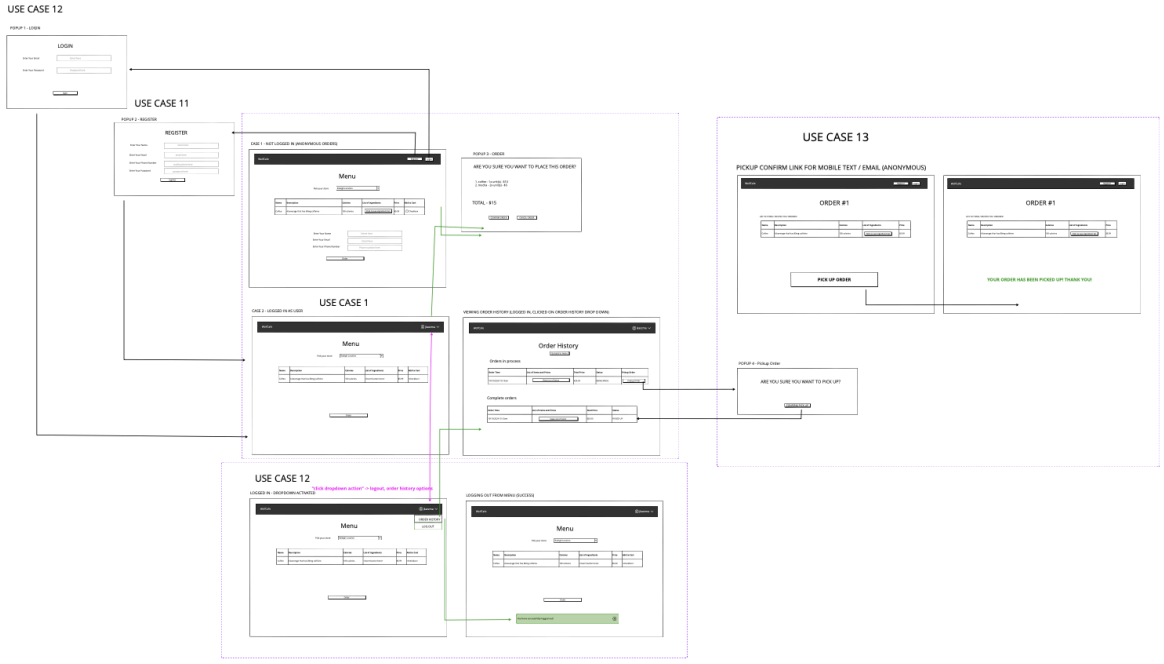
"Log in / log out"

Design Decisions (frontend, backend) - done

[**logging out**]: Given that logging out is only applicable to logged in users, we consider that this button is only visible as a potential option to people with the logged in state. As such, we decided to put this in a dropdown in the top right corner underneath their order history as a button they can click to disassociate with their current account. [**logging in**]: We decided on a simple intuitive interface for general user logins, a pop-up that shows when the registration button is clicked on our main menu page. This simplicity eliminates user confusion and given that there's only one place to register at, it improves the UX of the page. Popups will be instantiated as a callback from the button within React, so they will stay within the same JSX file as the main menu page ('**menu.jsx**'). [**both**]: We decided to store the user's current status in both a React state, as well as a **cookie stored in the user browser**. This allows for the user to be able to refresh the page and maintain their current logged in / logged out status, which promotes much better **ease of use**. Given that only general users can register from this place, we do not have to consider the Role aspect of this use case. Rather, we can focus on how we handle users generally. We store all of our users in a database table managed by a general repository ('**UserRepositor**y' class). We manage all of this in the backend through an authentication controller we designed (denoted '**AuthControlle**r' class), which allows for the registration, handling of authentication and deleting of users. Adjacently, we also consider we need a service to handle more complex operations not suited for being included directly in the controller class. We denote this as our '**AuthServiceImpl'** class, which contains several helper methods for the aforementioned class. We also had to consider how this would affect our frontend React components. This does affect our database, but given we are using Hibernate (our ORM tool) to map objects to our relational database (**MySQL**), our DTO (**'UserDto**') and entity object ('**User**'), allow us to easily manage these entries in our service implementations. In **Hibernate**, we will need **many-to-one relationships** as users will only have one role, and roles contain infinitely many users.

Frontend Elements

Original Design **(SAME AS UPDATED DESIGN)**



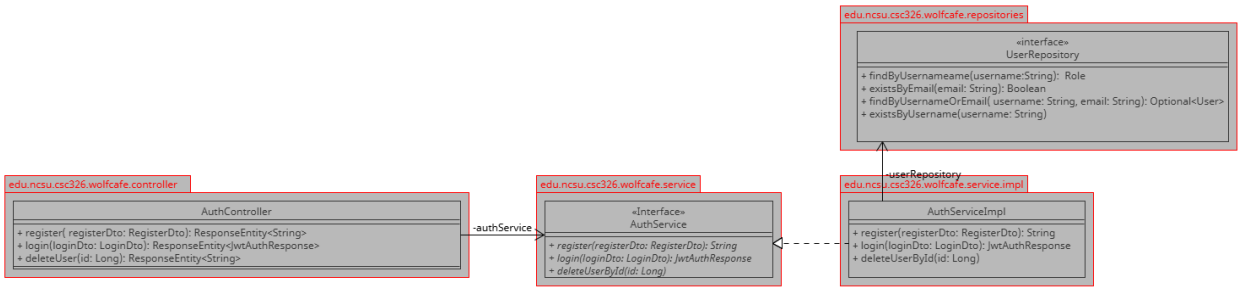
Changelog

- Since it's iteration 1A we did not change anything

Backend Elements

Class Diagrams -

Original Design (**SAME AS UPDATED DESIGN**)

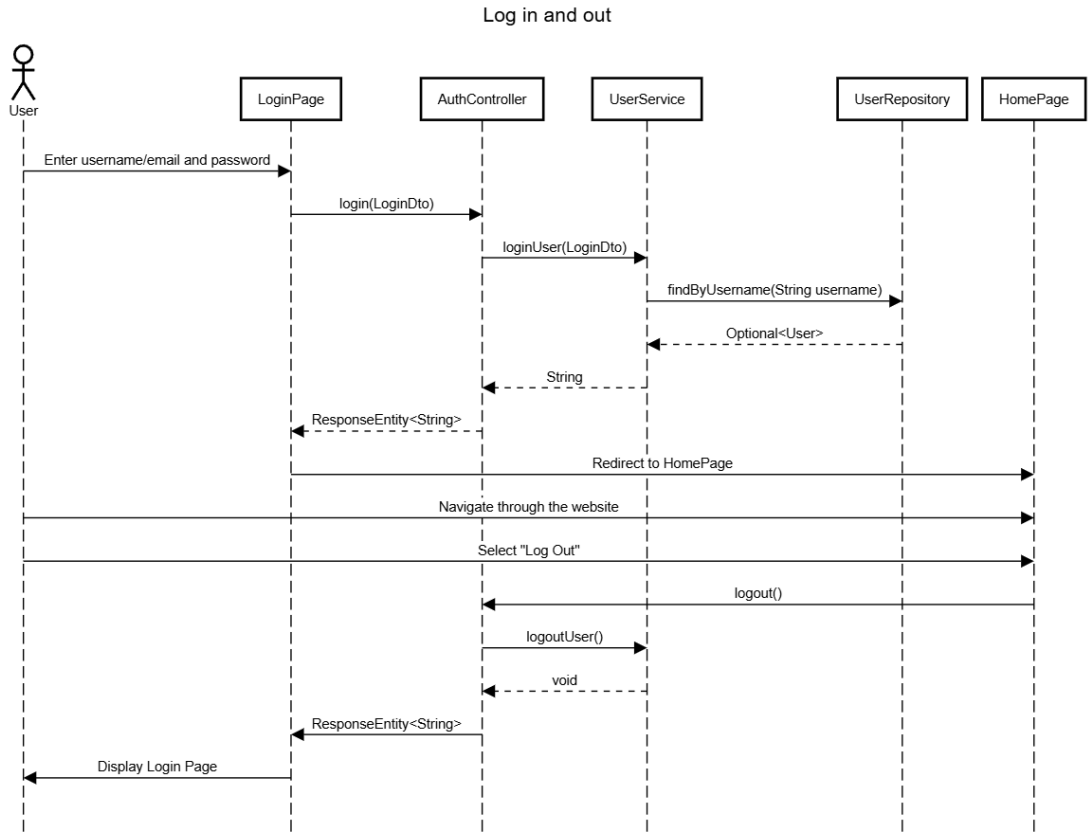


Changelog

- Since it's iteration 1A we did not change anything

Sequence Diagram:

Original Design (**SAME AS UPDATED DESIGN**)



Changelog

- Since it’s iteration 1A we did not change anything